

TECHNOLOGY STACK

ITC-Rec Engine

Build the MVP on the stack that also scales — add layers later, don't rewrite

The MVP stack **is** the expandable stack. “Better and more expandable” for this product is not a different base — it is the same core (Python / FastAPI / PostgreSQL / React) with scaling layers added around it later (a job queue, object storage, cache, multi-tenancy, real infrastructure). FastAPI and Postgres run at large scale in production today; you don't grow out of them, you grow into the layers beside them. So the job now is not picking future tech — it is building a few **seams** so each swap is a one-file change later, not a rewrite.

How to read the table

- **MVP (build now)** is what you implement for the first testable version.
- **Platform (later)** is what the same layer becomes at scale — additive, not a replacement.
- **Seam to build now** is the small abstraction in the MVP that makes the later swap painless.

Stack by layer

Layer	MVP (build now)	Platform (later)	Seam to build now
API framework	FastAPI	FastAPI (unchanged) + API gateway	None — already scales
Database	Postgres, single instance	Postgres + PgBouncer, read replica, partition by org_id / tax_period	Alembic migrations + org_id on every table from day 1
Heavy matching (M2)	runs inline in the request, pandas in-memory	runs on a worker off a queue (Celery / Dramatiq + Redis, or Temporal)	M6 orchestrator is one function run_reconciliation(run_id) — never logic inside the route handler
Big-data processing	pandas / openpyxl	deterministic passes pushed into SQL joins; Polars / DuckDB for fuzzy	M2 takes an iterable / frame via a thin adapter
File storage	local disk	S3 / R2 / MinIO	Storage interface (put / get / url) — local impl now
Auth	email + password JWT, single org	IdP (Clerk / Keycloak / Cognito), SSO, RBAC	everything behind one get_current_user() dependency
Multi-tenancy	one org (but org_id present)	many tenants; Postgres row-level security or schema-per-tenant	a tenant-scoping dependency that filters all queries by org_id
Cache	none	Redis (sessions, dashboard aggregates, rate limits)	add when needed — no seam required
Frontend	React + Vite SPA	React (unchanged); Next.js only if SSR / marketing is needed	typed API client generated from FastAPI's OpenAPI
External connectors	file upload only	GSP / GSTN, ERP / Tally, email / WhatsApp	LedgerSource interface — upload is just the first impl
Deploy	Docker Compose on Render / Railway / Fly	managed K8s / ECS + Terraform + managed Postgres	Dockerize + 12-factor config (env vars) from day 1
Observability	structured logging	+ Sentry + OpenTelemetry traces + metrics	a correlation / run id threaded through logs

The two seams that actually matter

Get these wrong and you pay for it later; get them right and everything else is additive.

1. Async-ready matching

The matching engine is the one thing that genuinely won't survive scale running inside an HTTP request (100k–500k rows × many tenants). You don't need a queue now — you need M6 written so that “do the reconciliation” is a plain function the route calls today and a worker calls later. In the MVP, invoke it via FastAPI `BackgroundTasks`; swapping to Celery later touches one line.

2. `org_id` everywhere + a scoping dependency

Retrofitting multi-tenancy onto a single-tenant schema is the classic painful migration. Put `org_id` on every table in the M0 schema; the only addition is making every query go through a tenant filter, so “one org” becomes “many orgs” for free.

What to deliberately NOT do yet

These are over-engineering traps — heavier, not more expandable:

- **Microservices** — a modular monolith with the M1–M10 boundaries scales far.
- **Kubernetes / Kafka / service mesh** before there's load to justify them.
- **Any NoSQL store** — reconciliation is joins; Postgres is the correct tool.
- **Rewriting matching in Rust / Go** before profiling proves Python + Polars / DuckDB isn't enough.